Moove Education Platform

System & Architecture Design Document (SDD)

Version 3.0

July 19, 2026

Prepared By:
Moove

This product includes software from the Open edX project, licensed under Apache 2.0.

July 20, 2026

Contents

0.1 System Overview

Moove Education Platform is an Open edX instance deployed using Tutor (v21.0.7) on Ubuntu 22.04. The architecture is container-based (Docker) and microservices-oriented. All components are orchestrated by Tutor and run inside Docker containers, with Caddy acting as the reverse proxy and TLS terminator.

0.2 Physical Architecture

The deployment consists of the following containers:

- **LMS (Learning Management System)**: Learner-facing operations (courses, enrollments, assessments). Runs uWSGI.
- **CMS (Course Management System)**: Educator authoring interface (Studio).
- **LMS Worker / CMS Worker**: Celery workers for background tasks.
- **MFE (Micro-Frontends)**: Serves modern frontend applications via Caddy.
- **Caddy**: Reverse proxy and automatic TLS (ports 80/443).
- **MySQL 8.4**: Primary relational database.
- **MongoDB 7.0**: Course structure and discussion data.
- **Redis 7.4**: Caching and Celery broker.
- **Meilisearch 1.8**: Search engine.
- **SMTP (Exim)**: Email relay.

0.3 Logical Architecture

0.3.1 Presentation Layer

- MFE apps (Account, Authn, Learner Dashboard) via Caddy. - Django templates for legacy parts.

0.3.2 Business Logic Layer

- Django/EdX-platform (Python) with plugins (Indigo theme, MFE). - Celery for asynchronous tasks.

0.3.3 Data Layer

- MySQL: Users, courses, grades, enrollments. - MongoDB: Course structures, discussions. - Redis: Session, cache, Celery broker. - Meilisearch: Search index.

0.4 System Architecture and Data Flow

0.4.1 Key Components

The Moove Education Platform consists of the following main components:

- **Reverse Proxy / TLS Termination:** Caddy handles all external HTTPS traffic, terminates TLS, and routes requests to the appropriate internal services.
- **Learning Management System (LMS):** Django application that serves learner-facing pages, handles enrollments, assessments, and grading.
- **Course Management System (CMS):** Django application (Studio) used by educators to author and manage courses.
- **Micro-Frontends (MFE):** React-based frontend applications (Account, Authn, Learner Dashboard) served via Caddy.
- **Background Workers:** Celery workers for asynchronous tasks (email, grading, analytics).
- **Primary Databases:** MySQL for user, course, and grade data; MongoDB for course structure and discussion data.
- **Caching and Messaging:** Redis for session storage, caching, and Celery broker.
- **Search Engine:** Meilisearch for full-text search across courses and content.
- **Email Relay:** SMTP (Exim) for sending system emails.

0.4.2 System Architecture Diagram

The following diagram illustrates the high-level system architecture, showing how components interact and the network segmentation used to isolate services.

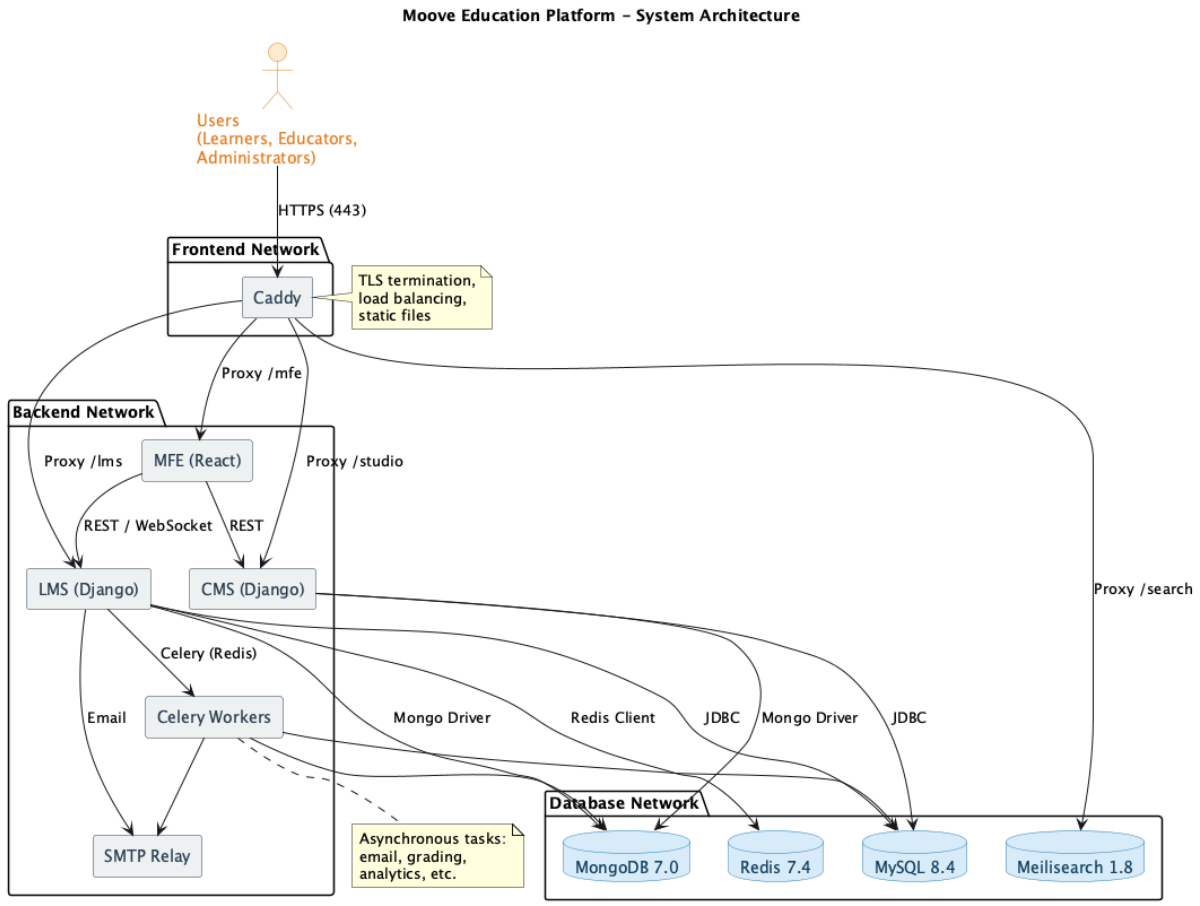

Figure 1: System Architecture Diagram showing Caddy, LMS, CMS, MFE, databases, and supporting services.

0.4.3 Data Flow Diagram (DFD)

The Data Flow Diagram shows how data moves through the system, from user interactions to storage and retrieval.

Context Diagram (Level 0)

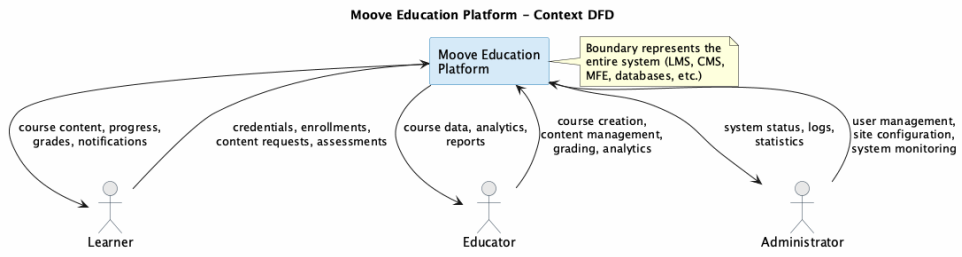

Figure 2: Context DFD showing external entities (Learner, Educator, Administrator) and the system boundary.

Level 1 DFD

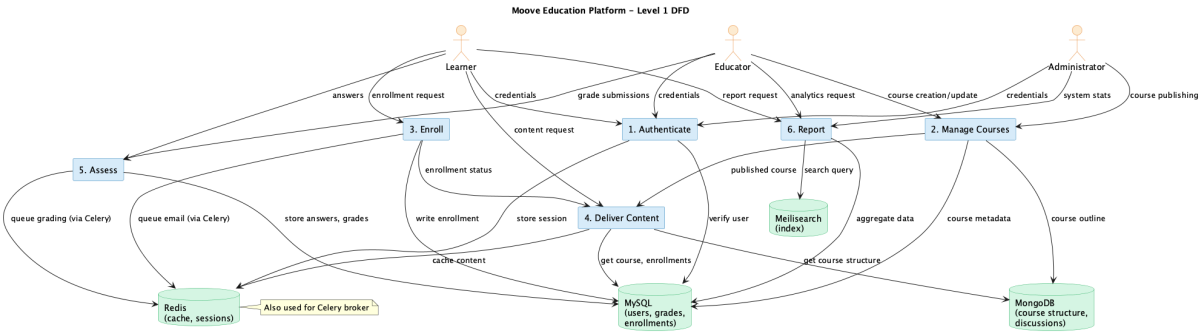

Figure 3: Level 1 DFD showing main processes: Authentication, Course Management, Enrollment, Content Delivery, Assessment, and Reporting.

0.4.4 Data Flow Explanation

The system processes data as follows:

- **Authentication:** User credentials are submitted via the MFE or LMS. Caddy passes the request to the LMS, which validates against MySQL and returns a session token (JWT).
- **Course Management:** Educators create course structures in the CMS. Data is stored in MySQL (course metadata) and MongoDB (course outline, XBlock content). When published, the LMS reads this data to render course content.
- **Enrollment:** When a learner enrolls, the LMS writes the enrollment record to MySQL and triggers a Celery task to send a confirmation email.
- **Content Delivery:** Learners request course pages; the LMS fetches the course structure from MongoDB, renders the content (using Django templates or MFE), and serves it via Caddy.
- **Assessments:** Learner answers are sent to the LMS, validated, and stored in MySQL. Grade calculations are performed asynchronously by Celery workers.
- **Search:** Meilisearch indexes course content and discussions. Search queries are routed through Caddy to the Meilisearch API.
- **Notifications:** The system generates notification events (e.g., discussion replies, assignment due dates) via Redis and Celery, sending emails through the SMTP relay.

0.5 Security Architecture (INSA Compliance)

This section describes the security controls implemented to meet INSA requirements. Detailed risk assessment and threat modelling are provided in the *Security Architecture & Threat Model* document.

0.5.1 Network Security

- Host firewall (UFW) is enabled and configured to allow only SSH (22), HTTP (80), and HTTPS (443) from external networks.
- Internal container networking is segmented into three Docker networks: `frontend`, `backend`, and `database`. The database network is internal and not exposed to the host.
- All unnecessary ports and services are disabled.
- Rate limiting and brute-force protection are provided by fail2ban.

0.5.2 Container Isolation

- Each service runs in its own container with minimal privileges.
- Tutor runs under a dedicated non-root user (`tutor`) with membership in the `docker` group.
- Resource limits (CPU, memory) are applied to containers to prevent resource exhaustion.

0.5.3 Secrets Management

- All sensitive configuration values (passwords, API keys, private keys) are stored encrypted and rotated regularly.
- Secrets are managed via Tutor's configuration system and not hard-coded in source code.
- JWT RSA private key is stored securely and rotated periodically.

0.5.4 Monitoring and Logging

- Centralised logging is implemented using the ELK stack (Elasticsearch, Logstash, Kibana).
- Security events (login failures, privilege changes) are logged and monitored.
- Weekly vulnerability scanning of container images is performed using Trivy.

0.5.5 Compliance

- The architecture is designed to meet CMCSRS requirements for access control, cryptography, and incident response.
- Annual security audits and accreditation are planned.
- All security controls are documented and tested.

For a complete threat model and risk assessment, refer to the *Security Architecture & Threat Model* (Security_OpenEdX.tex).

0.6 UML Diagrams

0.6.1 Use Case Diagram

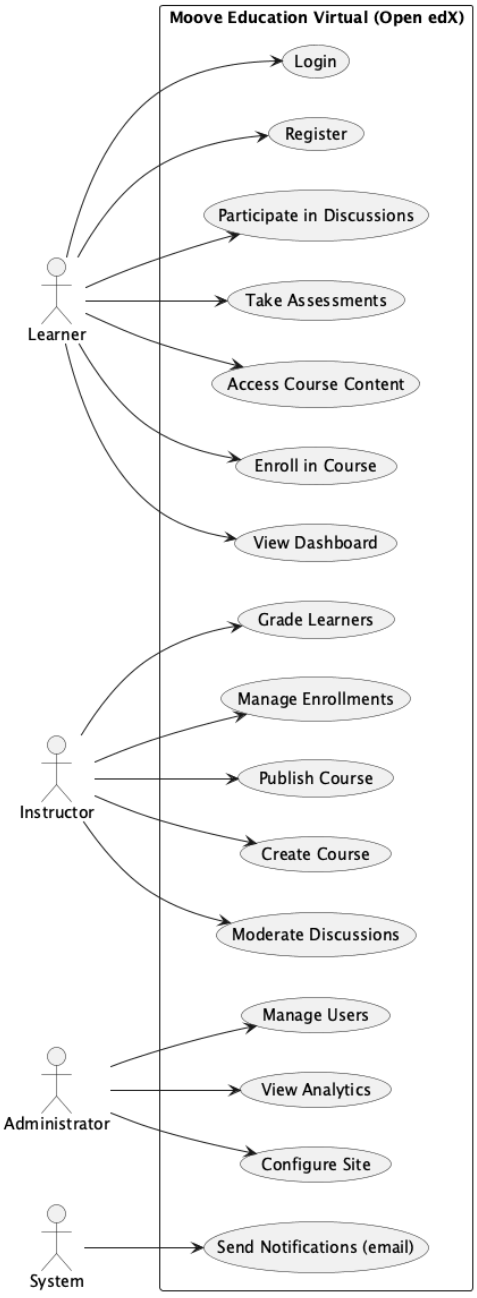

Figure 4: Use Case Diagram showing Educator and Learner interactions.

0.6.2 Class Diagram

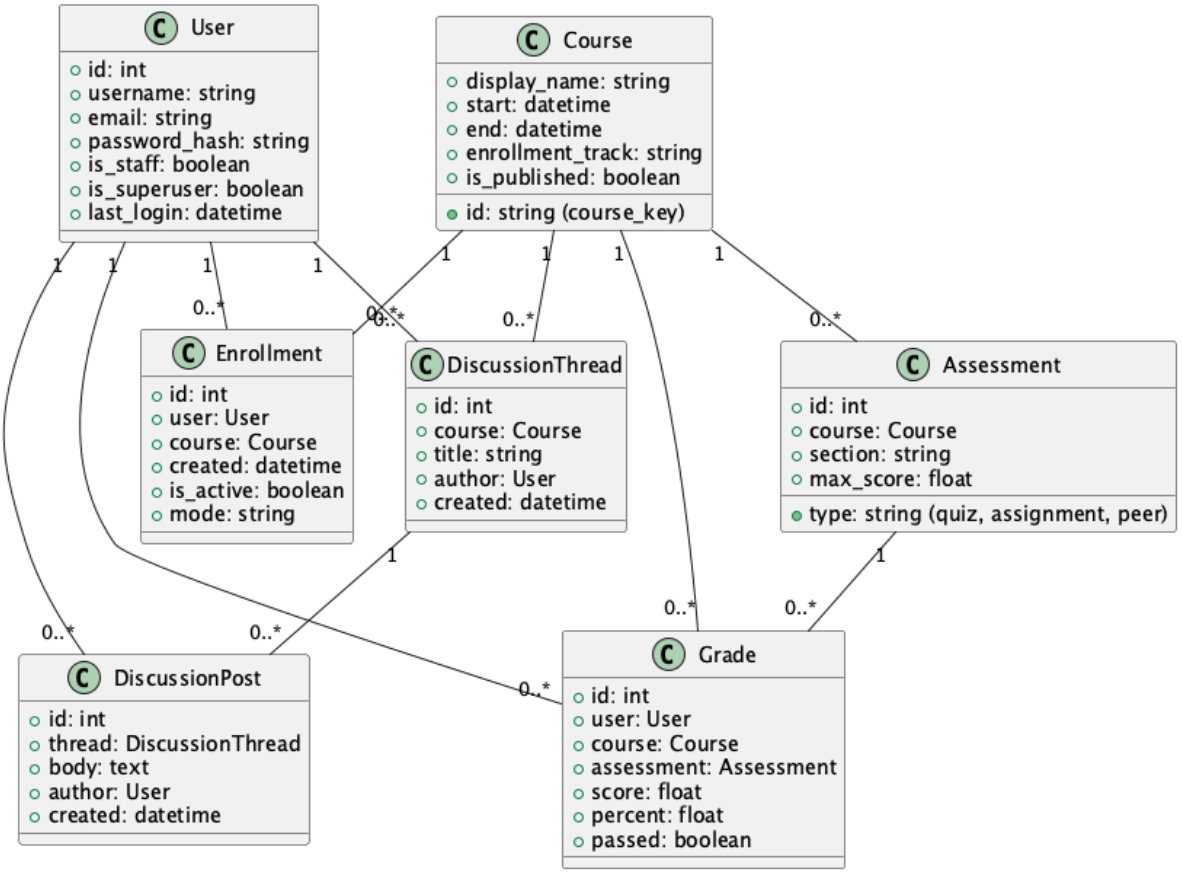

Figure 5: Core entities: User, Course, Enrollment, Assessment, Grade, Discussion.

0.6.3 Sequence Diagram: Course Enrollment

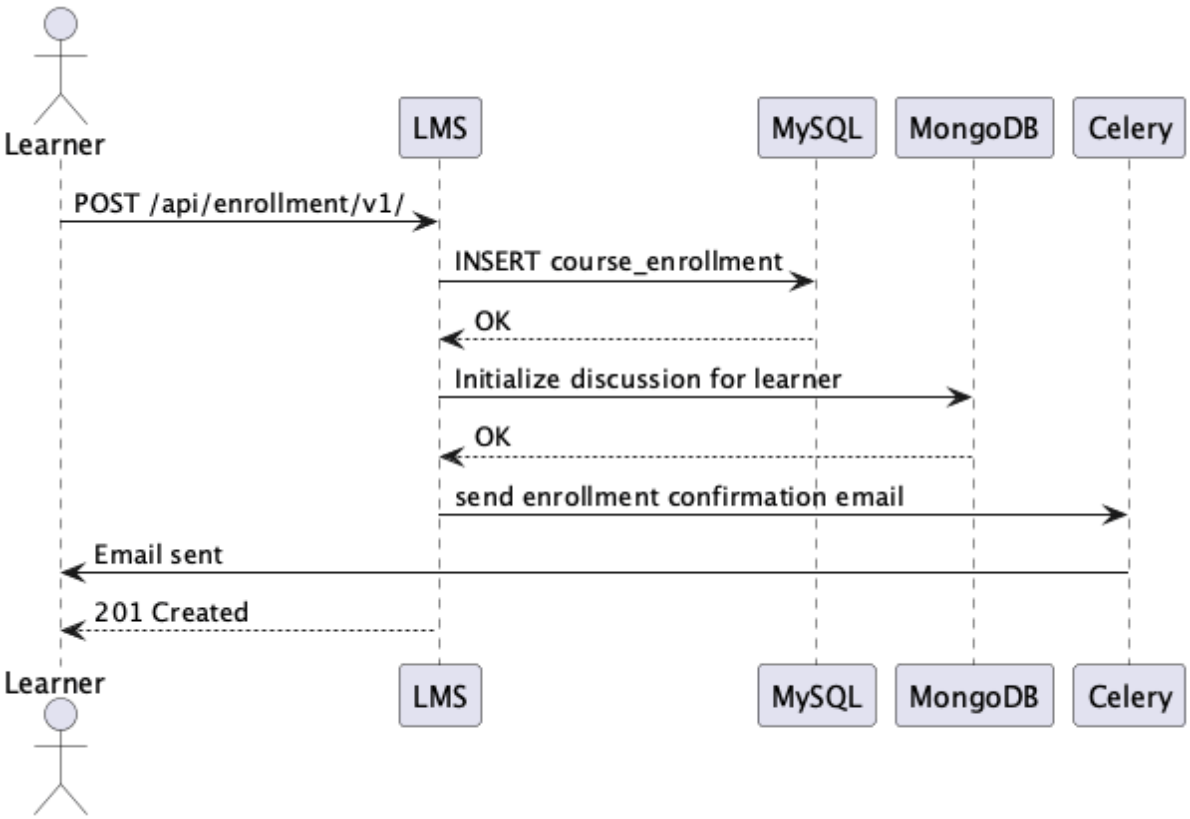

Figure 6: Learner enrollment flow.

0.6.4 Sequence Diagram: Course Creation (Educator)

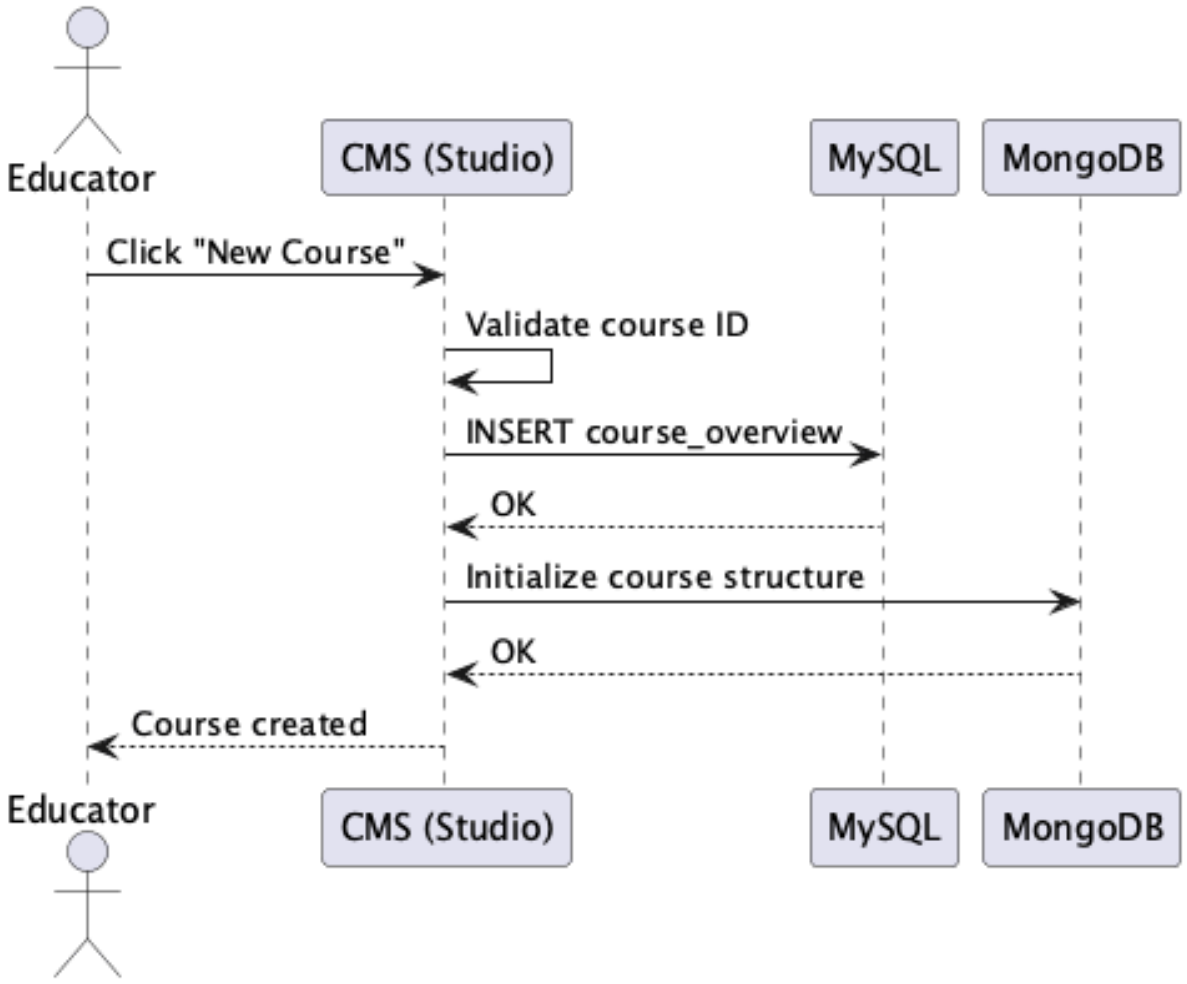

Figure 7: Educator course creation flow.

0.7 Module Interaction

Refer to the TDD for detailed component diagrams and API interactions.

0.8 References

- *Security Architecture & Threat Model – Moove Education Platform* (Security_OpenEdX.tex)
- CMCSRS v2.0
- FDRE National Cybersecurity Policy (2024)

Apache License 2.0 Attribution

This product includes software developed by the **Open edX Project** (`https://open.edx.org/`) licensed under the Apache License, Version 2.0 (`http://www.apache.org/licenses/LICENSE-2.0`).

Moove Education Platform is a derivative work of Open edX. No modifications have been made to the original license terms. A copy of the full license text is available in the root of the source repository and in Appendix A of this document.

.1 Full Apache License 2.0 Text